

PRESENTACIÓN

Clustering

K-Means, jerárquico y DBScan

Pablo Sciolla

Universidad de San Andrés — Analítica de Datos

Junio 2026

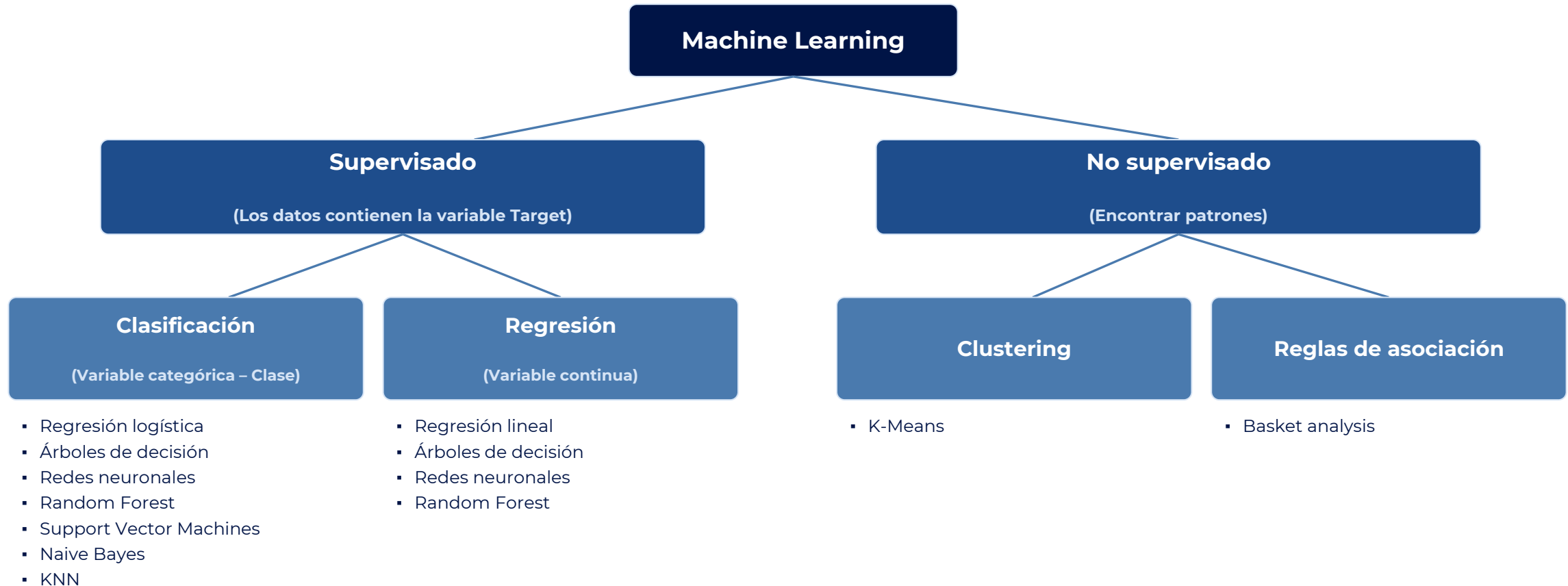
01

SECCIÓN

K-Means

02

Scope Machine Learning



Clustering

- Conjunto de técnicas para encontrar subgrupos o clusters dentro de un dataset.
- La idea es particionar las observaciones en diversos grupos de modo que:
 - Las observaciones dentro de un mismo grupo son similares entre ellas.
 - Las observaciones en diferentes grupos son diferentes entre ellas.
- Para que sea concreto, hay que definir qué significa que dos observaciones sean similares o diferentes.
- Una aplicación típica es la segmentación de clientes o usuarios para marketing.
- Existen varias técnicas; k-means es la más popular y la más sencilla.

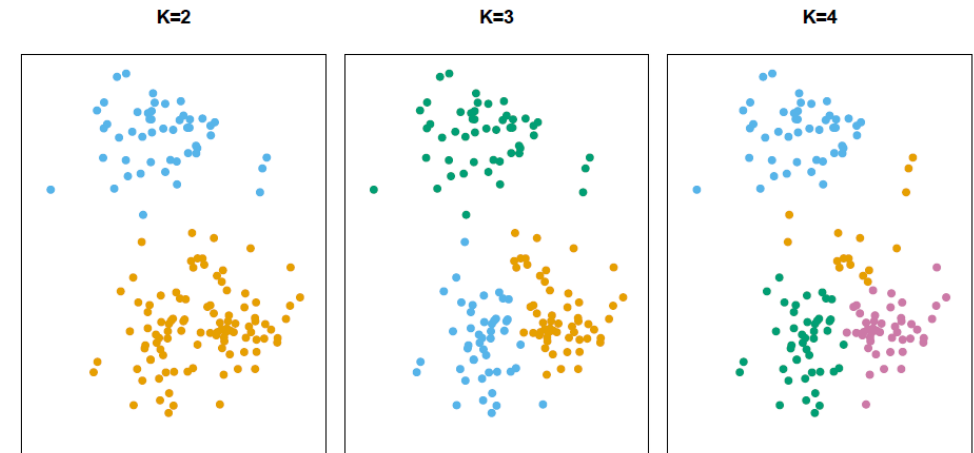
K-Means

K-means busca un número fijo de clústeres (k) en un conjunto de datos. Un clúster es una colección de puntos agrupados por ciertas similitudes. Los clusters satisfacen las siguientes propiedades:

$$C_1 \cup C_2 \cup \dots \cup C_K = \{1, \dots, n\}$$

$$C_k \cap C_{k'} = \emptyset \quad \text{para todo } k \neq k'$$

para todo $k \neq k'$



K-Means

Si la observación i está en el cluster k , entonces $i \in C_k$

La idea detrás de k-means es que un buen clustering es aquel en el que la varianza intra-cluster es la menor posible. La varianza intra-cluster para el cluster C_k mide cuánto difieren entre sí las observaciones dentro del cluster. Queremos resolver:

$$\min_{C_1, \dots, C_K} \left(\sum_{k=1}^K W(C_k) \right)$$

En palabras: queremos particionar las observaciones en K clusters de modo que la varianza total intra-cluster, sumada sobre todos los clusters, sea tan chica como sea posible.

K-Means

Para resolver el problema necesitamos definir

$$W(C_k)$$

Suma de las diferencias entre las características.

Existen varias formas, pero la más común es usar la distancia euclídea:

$W(C_k)$ es la with-in cluster variation.
Es el puntaje de dispersión dentro del cluster k.

$|C_k|$ es la cantidad de observaciones en el cluster k
 p es la cantidad de variables o parámetros
 i es la cantidad de observaciones

$$W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2$$

Comparar todos los elementos dentro del cluster

Que tan lejos esta el elemento i del dato i' en la característica k

En otras palabras, la variación intra-cluster para el cluster k es la suma de las distancias entre las observaciones del cluster k dividida por la cantidad de observaciones del cluster.

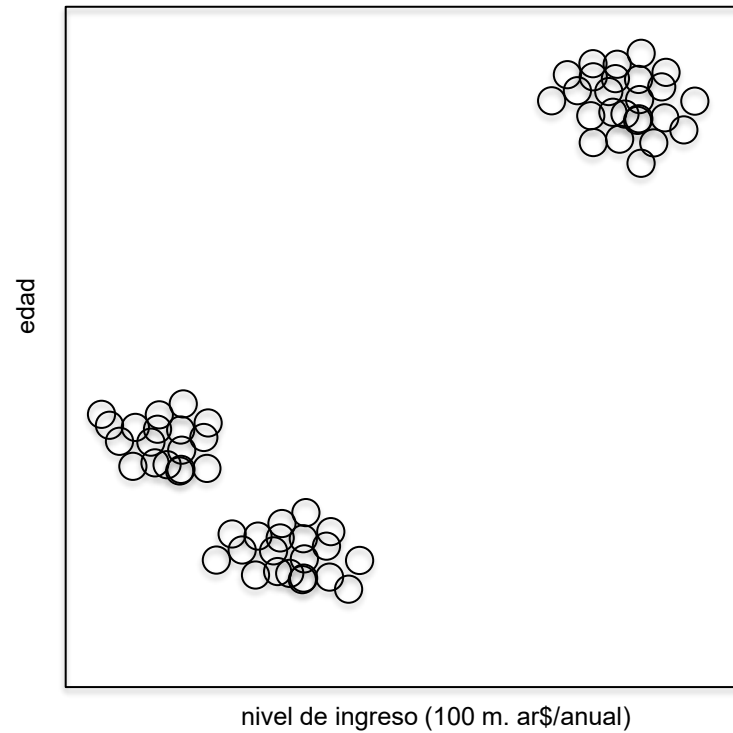
Resumiendo, el clustering k-means se puede representar con la siguiente fórmula:

$$\min_{C_1, \dots, C_K} \left(\sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$

K-Means — Aplicaciones

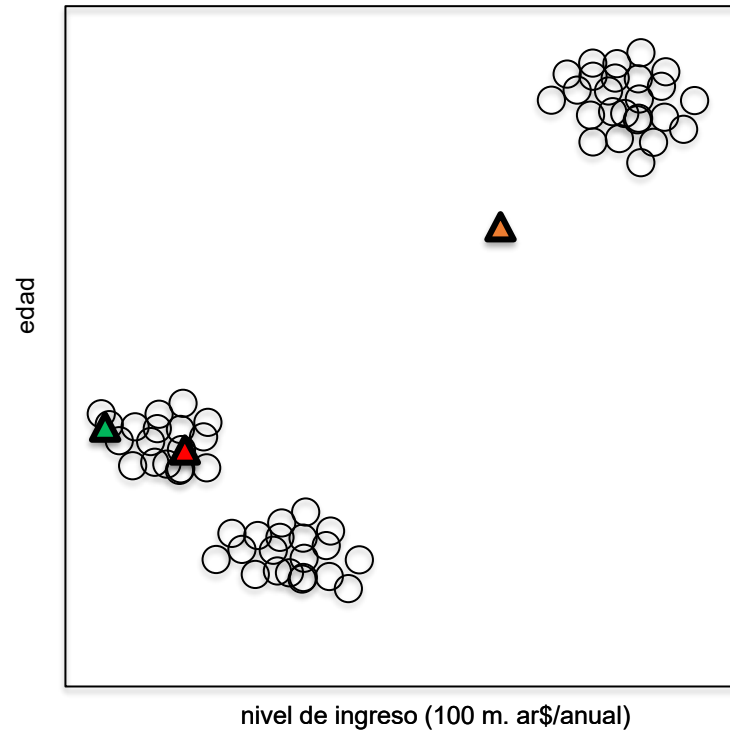
- Segmentación de clientes: características comunes de las personas para compartir campañas.
- Root cause analysis en delivery: agrupar ítems con issues (p. ej. incumplimiento de fecha) y extraer características comunes.
- Detección de fraude: agrupar movimientos con características similares.
- Identificación de fake news: examinar las palabras del corpus del artículo y agrupar.
- Análisis de documentos: búsqueda en el texto y agrupamiento por temáticas.
- Composición de equipos: detección de elementos comunes.
- Agrupamiento de imágenes por similitud: extraer características principales y agrupar en base a ellas.

K-Means — Algoritmo



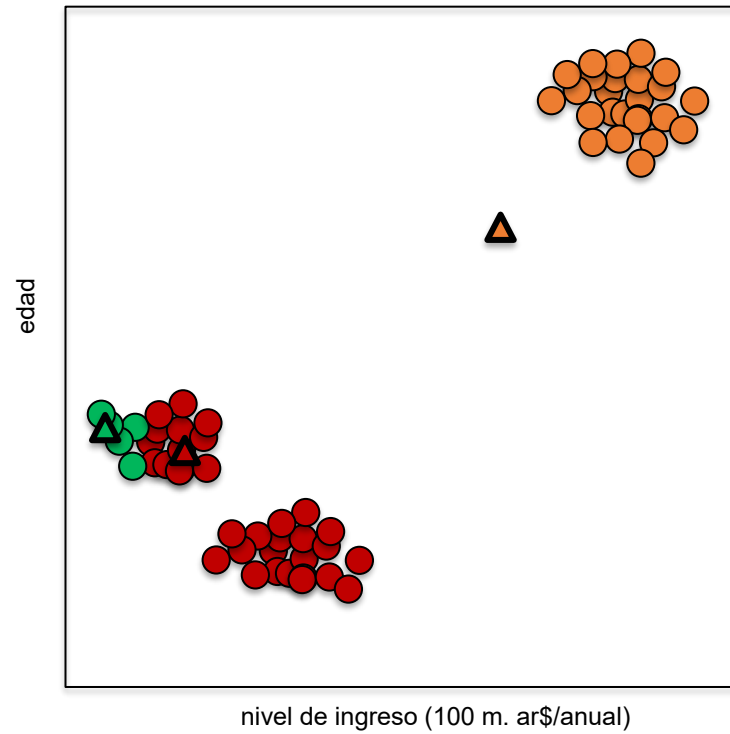
1° Definir la cantidad de clusters (k)

K-Means — Algoritmo



2° Asignar k puntos en forma aleatoria (los vamos a llamar centroides)

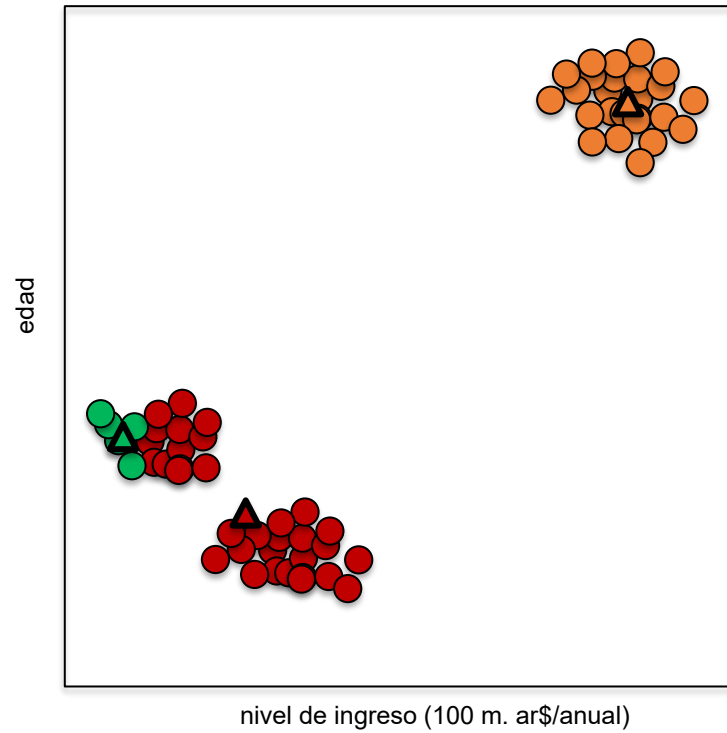
K-Means — Algoritmo



3° Calcular la distancia de cada punto a cada centroide.

4° Asignar cada punto al centroide más cercano.

K-Means — Algoritmo

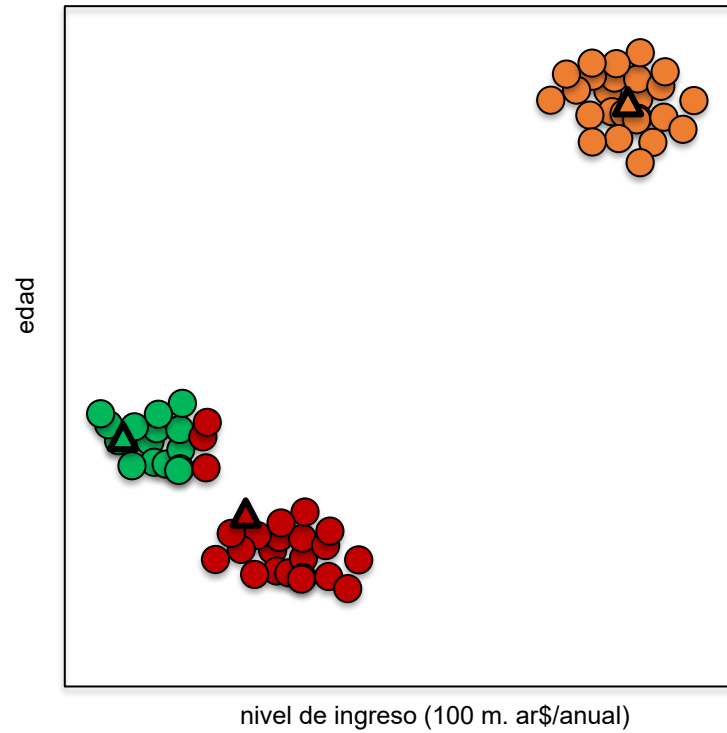


5° Recalcular la
posición del centroide

*** coordenadas del
centroide: promedio
de coordenadas de las
observaciones de ese
cluster ***

K - M e a n s

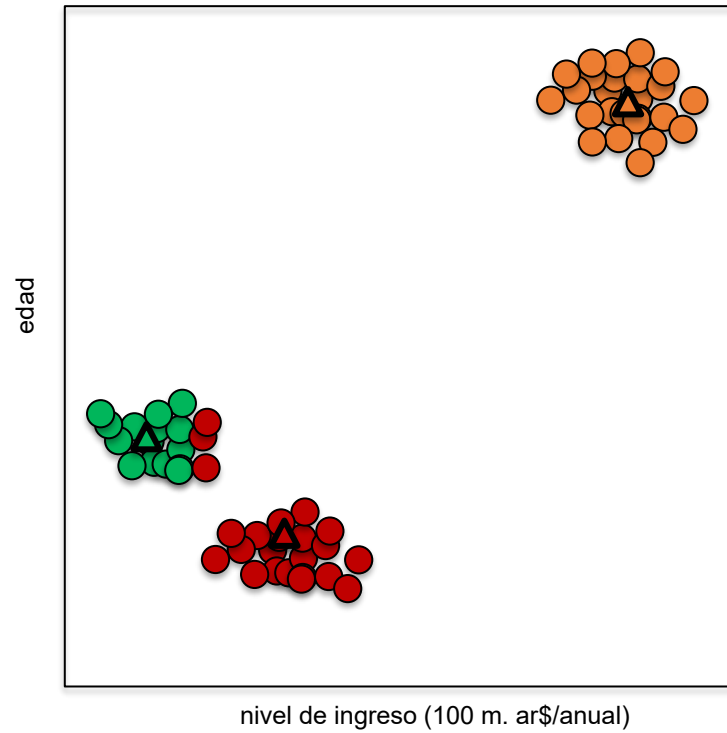
K-Means — Algoritmo



6° Reasignar cada punto de datos por cercanía en función de la nueva posición del centroide

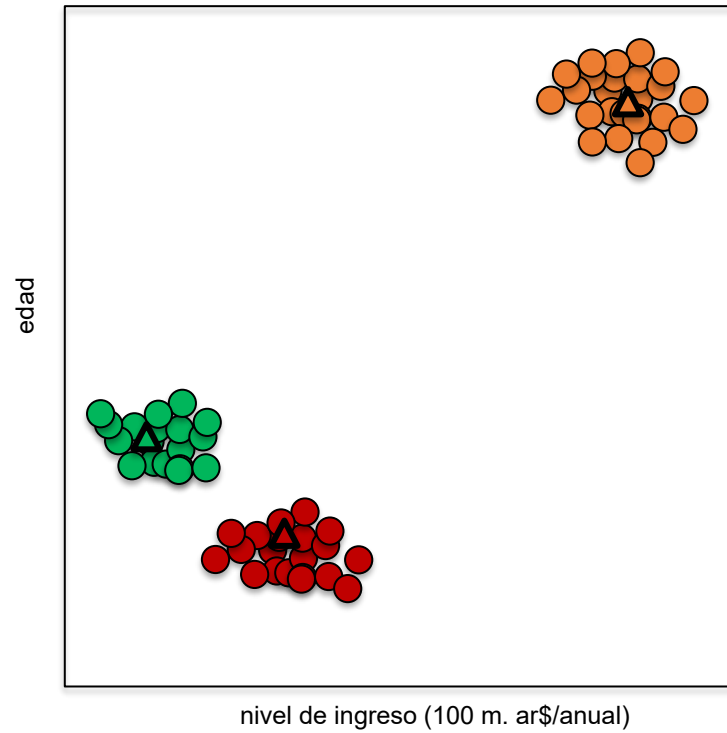
K - M e a n s

K-Means — Algoritmo



7° Recalcular la
posición del centroide

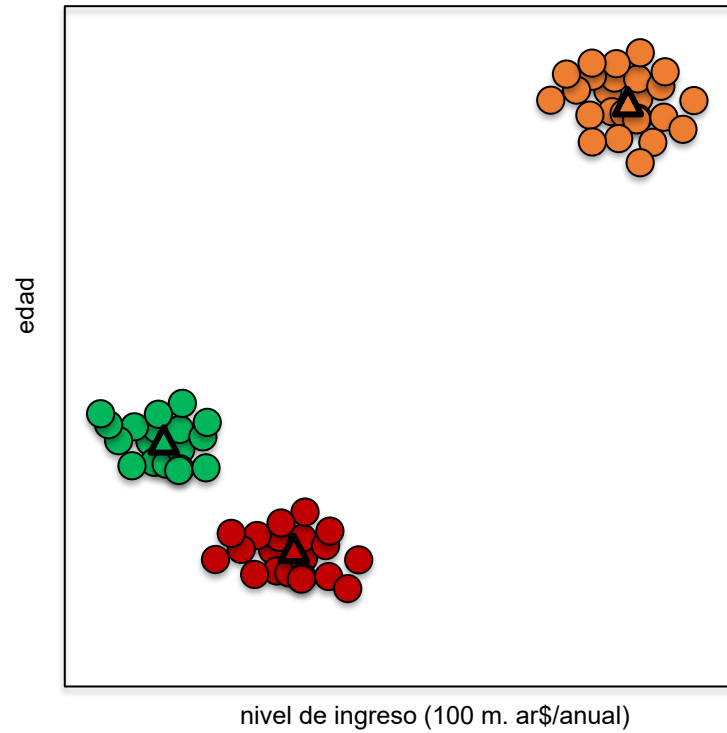
K-Means — Algoritmo



8° Reasignar cada punto de datos por cercanía en función de la nueva posición del centroide

K - M e a n s

K-Means — Algoritmo

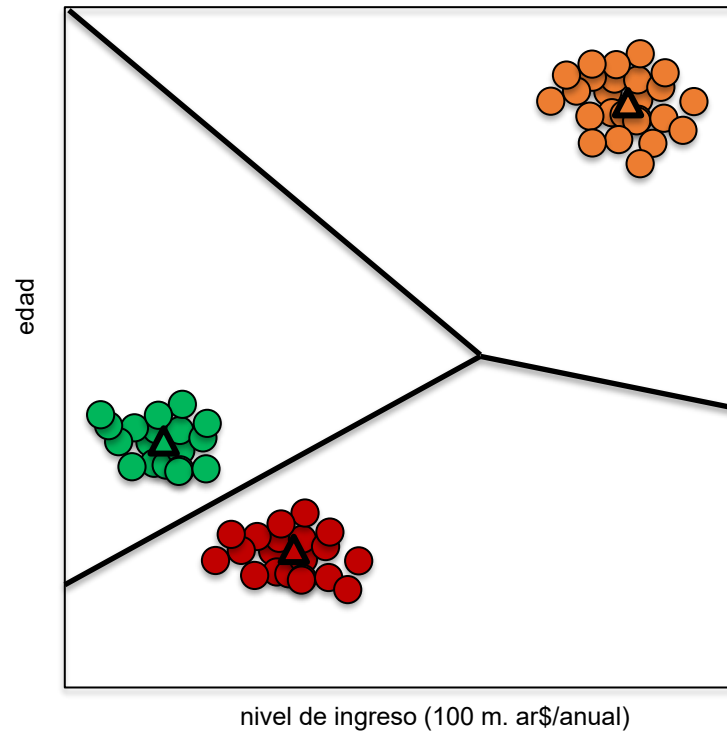


9° Recalcular la
posición del centroide

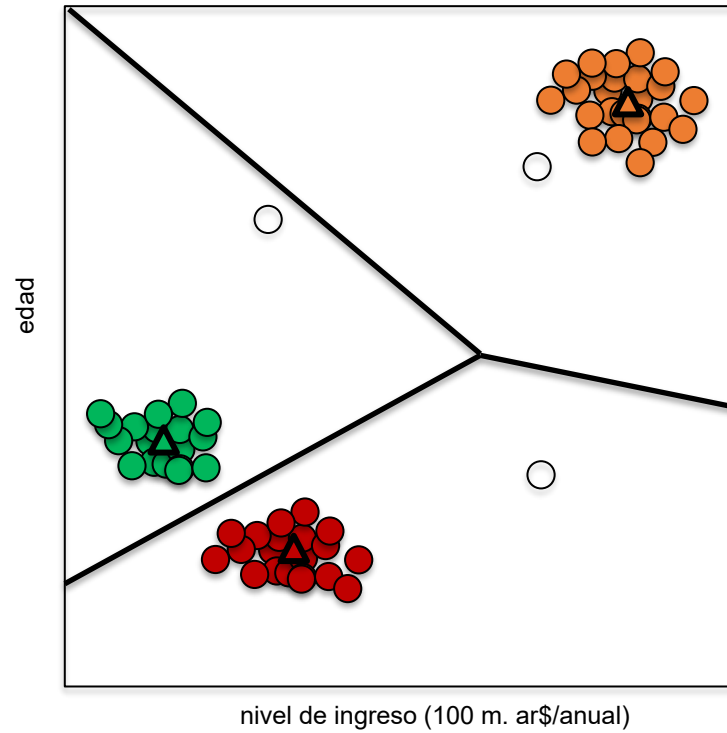
K-Means — Algoritmo

El cálculo de la posición del centroide y la reasignación de observaciones al centroide más cercano se repiten hasta que ya no se evidencian cambios.

K-Means — Algoritmo

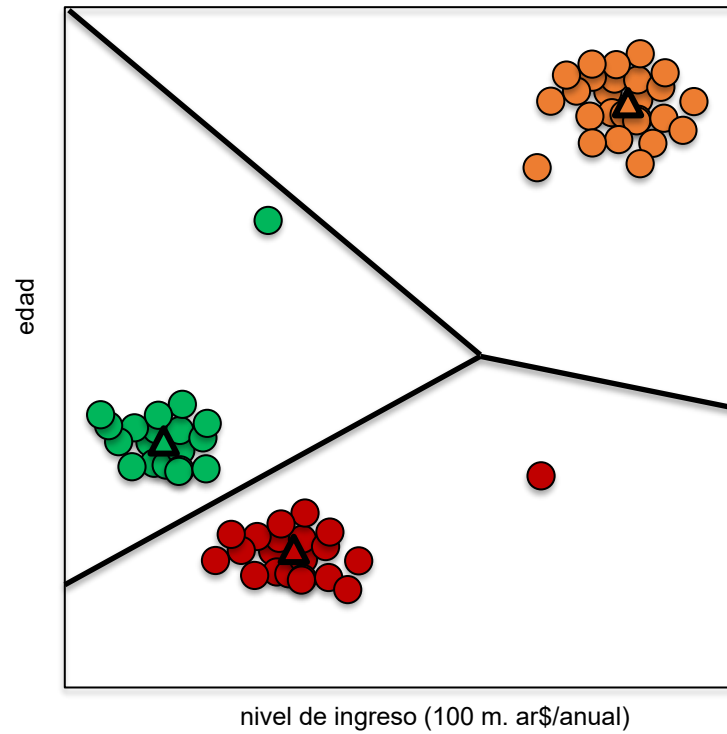


K-Means — Algoritmo



Nuevos puntos de
datos

K-Means — Algoritmo



Cada nuevo punto de datos es asignado al cluster correspondiente en función de la frontera

SECCIÓN

Validación

03

Validación

- Es muy difícil determinar si una partición (una configuración con k clusters) es buena o no.
- En 2 o incluso 3 dimensiones podemos apoyarnos en el gráfico, pero en dimensión más alta eso se vuelve imposible: por eso es crucial un análisis a posteriori de la configuración encontrada por el algoritmo.
- Lo que sí podemos hacer es comparar particiones entre sí para encontrar la mejor, pero ¿qué métrica usamos?

Validación

Medidas de cohesión

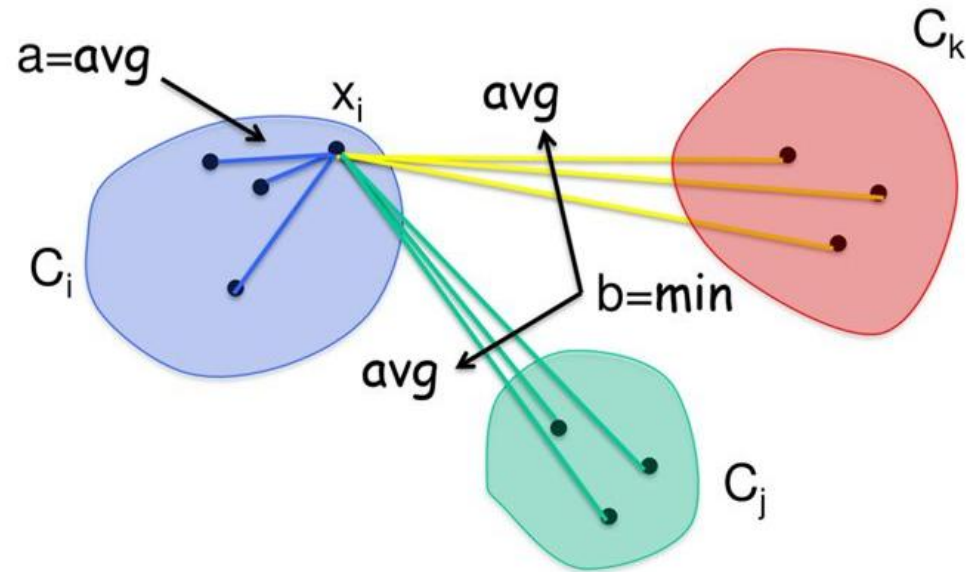
- Miden qué tan cerca están las observaciones dentro del mismo cluster.

Medidas de separación

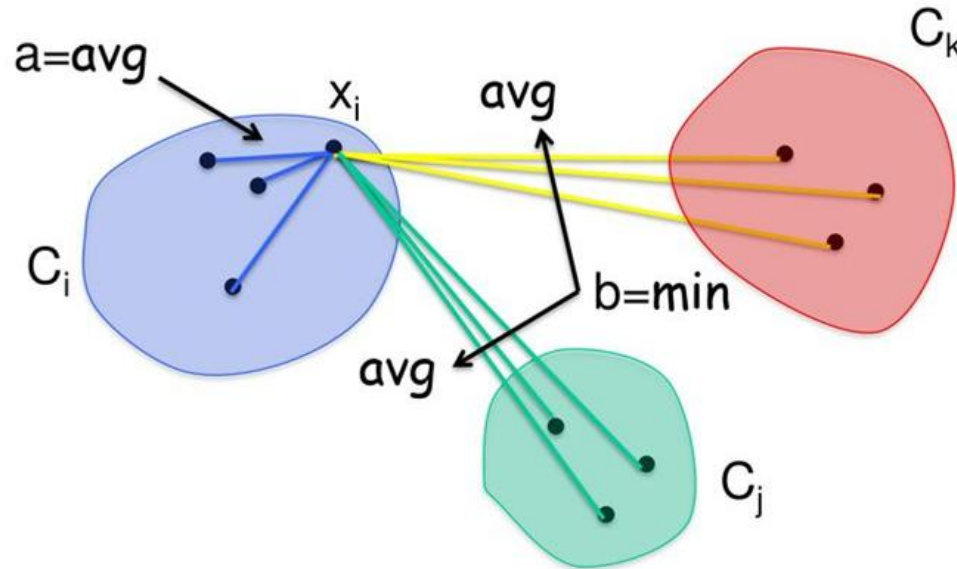
- Miden qué tan separados están los clusters entre sí.

Coeficiente de Silhouette

Combina una medida de cohesión y una de separación.



Coeficiente de Silhouette



- **Cercano a 1 (Excelente):** Significa que $b(i) \gg a(i)$. El punto está muy cerca de sus compañeros y lejísimos del grupo de al lado. El cluster está perfecto.
- **Cercano a 0 (Zona gris):** Significa que $a(i)$ es aproximadamente $b(i)$. El punto está en la frontera, a mitad de camino entre su grupo y el vecino. Podría pertenecer a cualquiera de los dos.
- **Cercano a -1 (Alerta/Error):** Significa que $a(i) \gg b(i)$. El punto está más cerca de los puntos del otro grupo que de los del suyo propio. El algoritmo probablemente lo asignó al cluster equivocado.

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

Que tan cómodo está el elemento x_i con elementos del mismo cluster.

$a(i)$ es la distancia promedio desde x_i a todos los demás puntos de su mismo cluster.

$$b(i) = \min_{k \neq i} \frac{1}{|C_k|} \sum_{j \in C_k} d(i, j)$$

Que tan incómodo está el elemento x_i con los elementos de todos los otros clusters.

$b(i)$ es la distancia al promedio al cluster vecino más cercano (por eso el min).

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Busca la distancia al vecino (b) sea gigante y que la distancia interna (a) sea diminuta

Coeficiente de Silhouette

1

Se calcula el coeficiente de Silhouette para cada una de las observaciones, luego del agrupamiento.

2

Se calcula el coeficiente de Silhouette de la partición promediando los coeficientes de todas las observaciones.

3

Se puede calcular el coeficiente de Silhouette de cada cluster promediando los coeficientes de las observaciones dentro de ese cluster.

Coeficiente de Silhouette — Segmentación Wahoo

Calculemos el coeficiente de Silhouette del cliente 593...

Cluster 0	Cluster 1	Cluster 2
125, 183, ..., 590, 593, ..., 1311, 1313	3, 159, 160, ..., 1308, 1310	161, 176, ..., 1297, 1299, 1301

1. Escalar los datos.
2. Calcular la distancia promedio entre 593 y el resto de las observaciones del cluster 0 — $a(i)$.
3. Calcular la distancia promedio entre 593 y las observaciones del cluster más cercano al 0 — $b(i)$.
4. Calcular el coeficiente de Silhouette.

Coeficiente de Silhouette — Segmentación Wahoo

Cálculos: [Google Colab](#)

Distancia promedio entre 593 y el resto de las observaciones del cluster 0 — a(i)

6.358

Coeficiente de Silhouette — Segmentación Wahoo

Cálculos: [Google Colab](#)

Distancia promedio entre 593 y las observaciones del cluster más cercano al cluster 0
— $b(i)$

6.2575

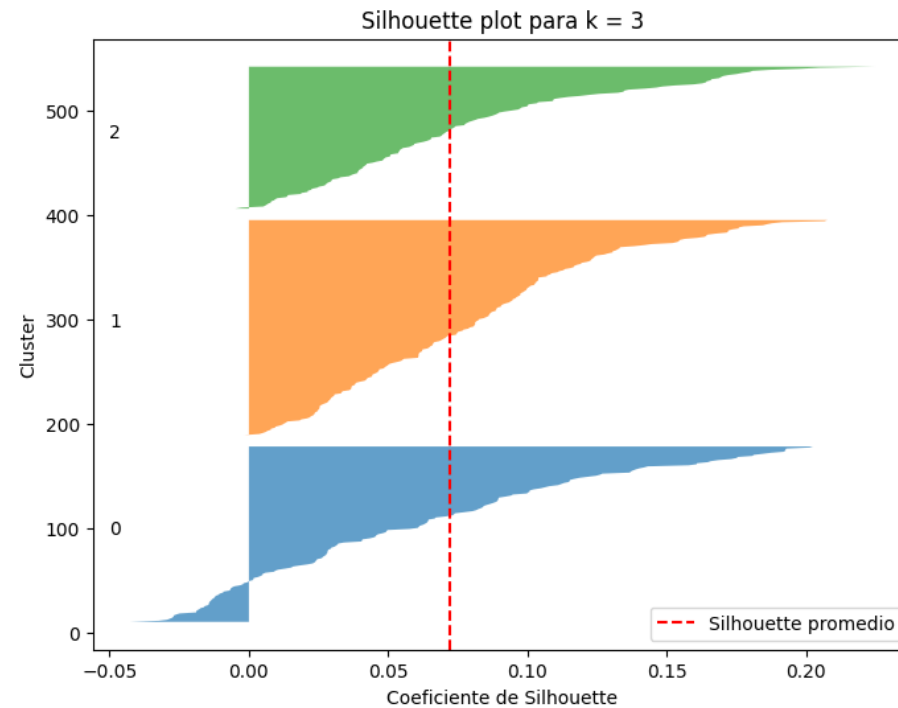
Coeficiente de Silhouette — Segmentación Wahoo

Cálculos: [Google Colab](#)

Coeficiente de Silhouette de CID 593: $(b - a) / a$

-0.015

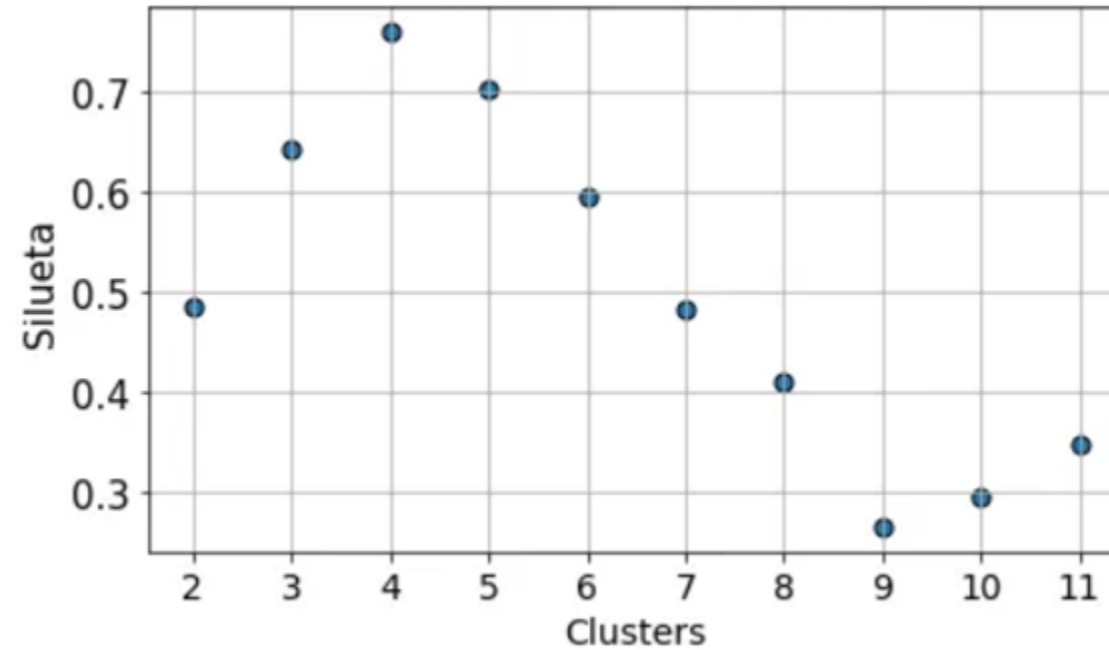
Coeficiente de Silhouette — Segmentación Wahoo



K - M e a n s

K-Means — Elección de k

Se evalúa el coeficiente de Silhouette para distintos valores de k y se elige el que lo maximiza.



SECCIÓN

Método del Codo

03

El problema: ¿cuántos clusters?

- K-means necesita que definamos k (la cantidad de clusters) antes de correr el algoritmo.
- Pero en la práctica casi nunca sabemos de antemano cuántos grupos hay en los datos.
- Si elegimos k muy chico, mezclamos grupos que en realidad son distintos.
- Si elegimos k muy grande, partimos grupos reales en pedazos artificiales.
- Necesitamos un criterio para elegir un k razonable. El método del codo es el más usado.

Concepto

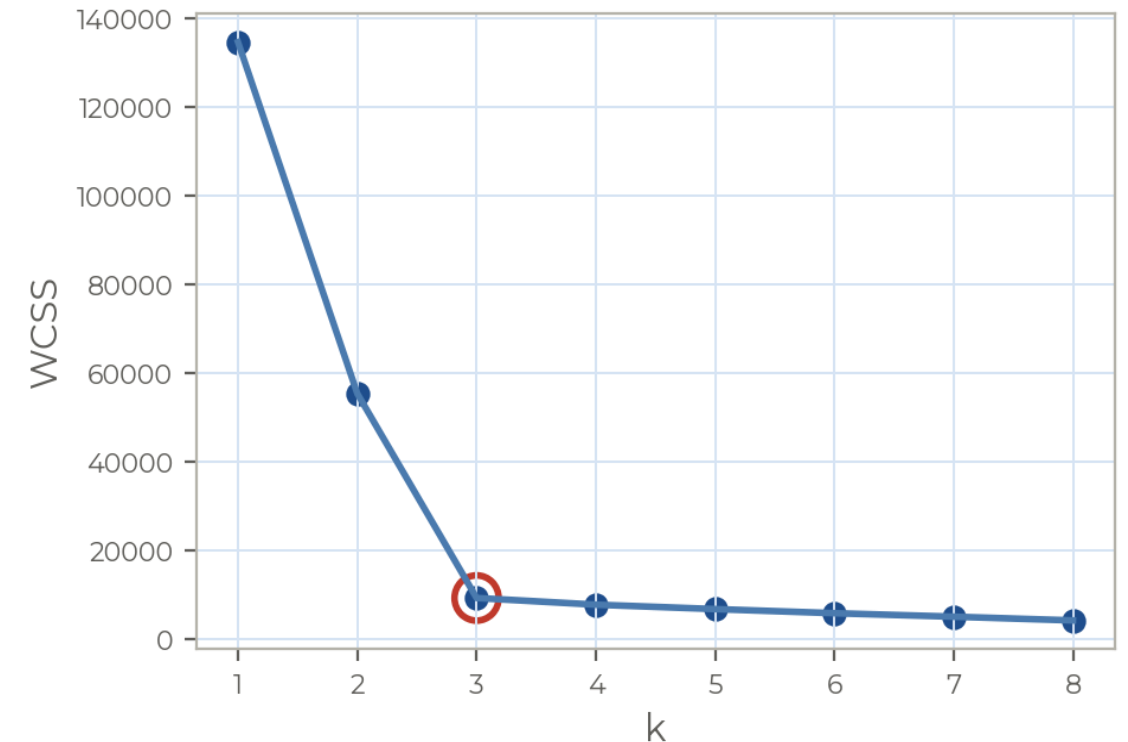
¿Qué es el método del codo?

La idea es correr k-means para un rango de valores de k y medir, para cada uno, qué tan compactos quedan los clusters mediante la WCSS (within-cluster sum of squares), también llamada inercia:

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Es la suma de las distancias al cuadrado de cada punto a su centroide.

A medida que aumenta k, la WCSS siempre baja; lo que buscamos es el punto donde deja de bajar de forma pronunciada: el codo.



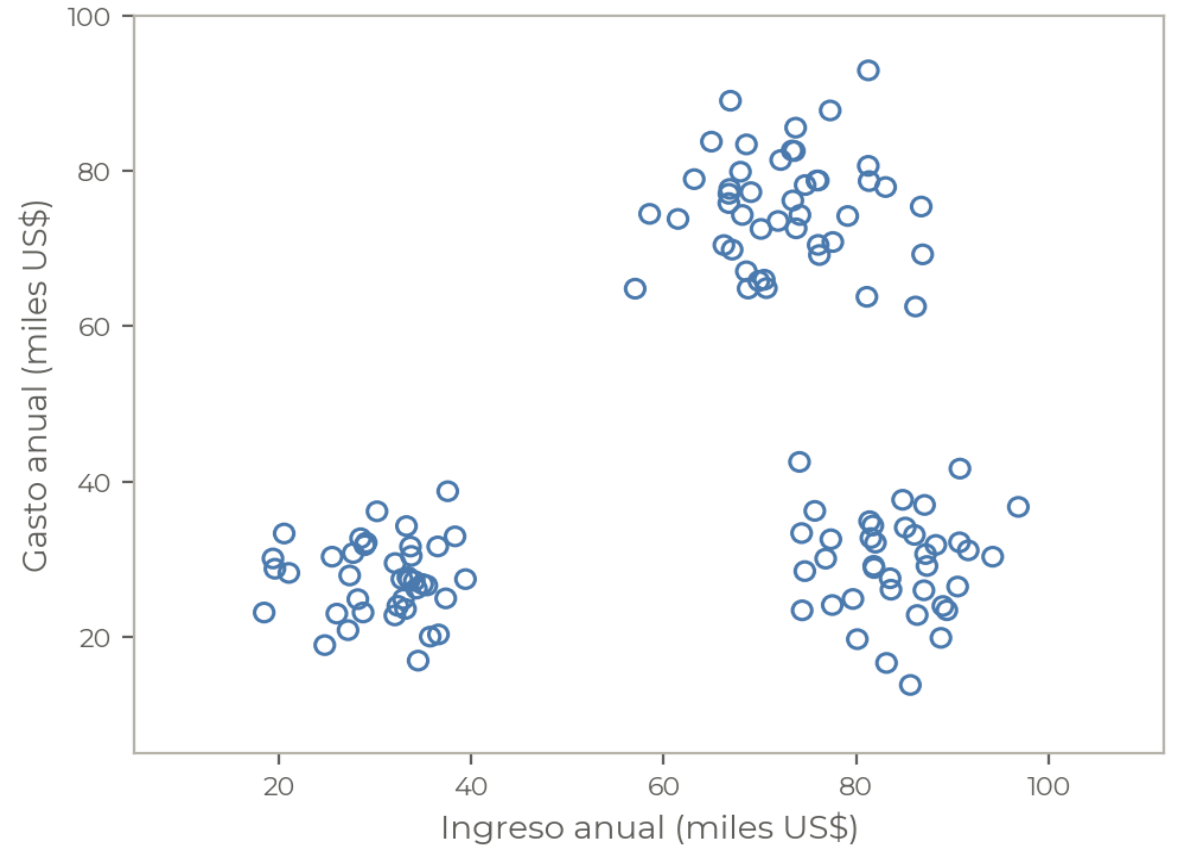
El método paso a paso

- 1 Elegir un rango de valores de k a evaluar (por ejemplo, de 1 a 8).
- 2 Correr k-means para cada valor de k .
- 3 Calcular la WCSS (inercia) resultante de cada partición.
- 4 Graficar la WCSS en función de k .
- 5 Identificar el codo: el k a partir del cual la WCSS deja de bajar de forma marcada.

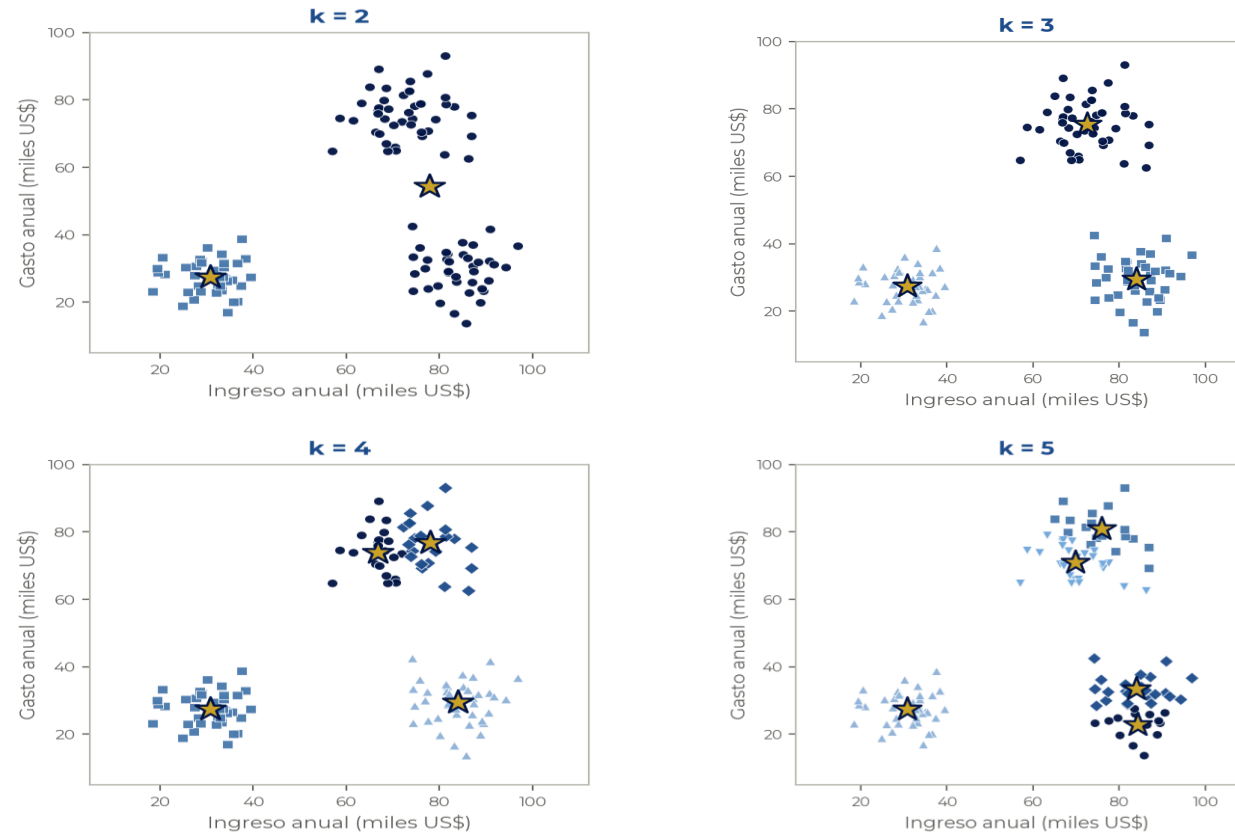
Ejemplo

El dataset: segmentación de clientes

Tenemos 123 clientes descritos por dos variables: su ingreso anual y su gasto anual (en miles de US\$). Queremos segmentarlos, pero no sabemos en cuántos grupos. A simple vista parecen distinguirse algunos conglomerados, ¿pero cuántos exactamente?



Paso 1-2: corremos k-means para varios k



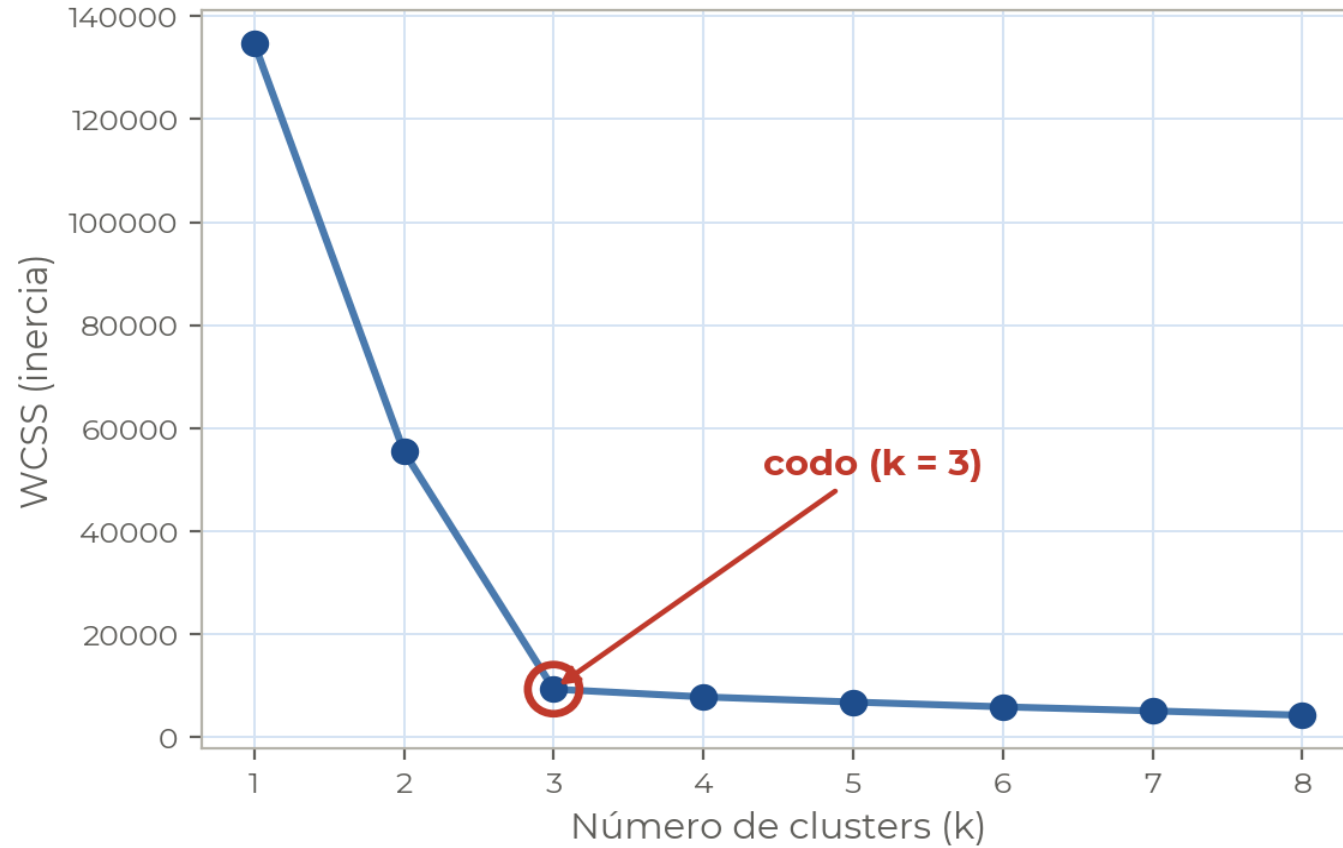
Paso 3: calculamos la WCSS de cada partición

Para cada k registramos la inercia y cuánto se reduce respecto del k anterior. La caída se desploma después de k = 3.

k	1	2	3	4	5	6	7	8
WCSS	134.718	55.443	9.314	7.808	6.813	5.884	5.098	4.254
Reducción	—	58,8%	83,2%	16,2%	12,7%	13,6%	13,4%	16,5%

Valores de WCSS en miles. El salto de 55.443 a 9.314 (–83,2%) es la última gran mejora; sumar un cuarto cluster casi no reduce la inercia.

Paso 4-5: graficamos y buscamos el codo



La curva baja muy rápido de $k=1$ a $k=3$ y después se aplan.

Ese quiebre —donde agregar un cluster más deja de aportar— es el **codo**. Aquí está claramente en **$k = 3$** .

Ejemplo

Conclusión: elegimos $k = 3$

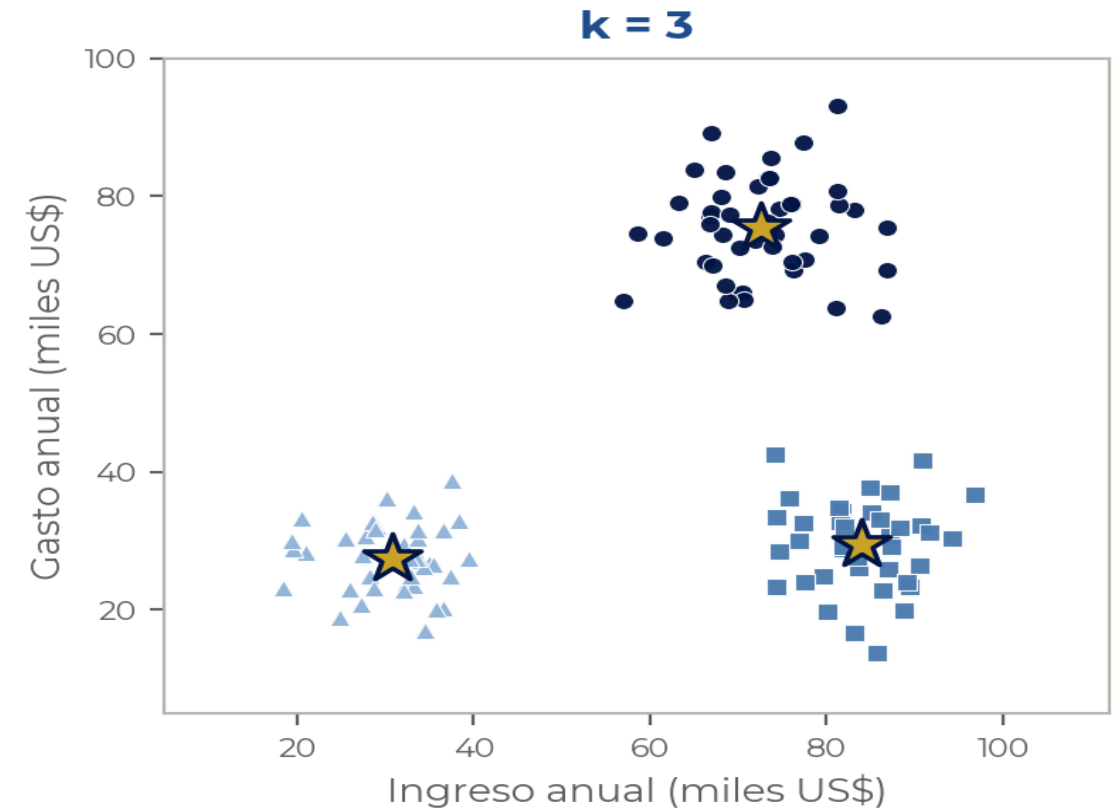
El codo nos dice que tres clusters capturan la estructura de los datos sin sobre-segmentar.

Los tres grupos tienen una lectura de negocio clara:

Ingreso bajo · gasto bajo clientes de bajo valor o iniciales.

Ingreso alto · gasto alto clientes premium, foco de fidelización.

Ingreso alto · gasto bajo oportunidad: potencial no capturado.



Ventajas y limitaciones

Ventajas

- Simple e intuitivo de explicar.
- Computacionalmente barato.
- Funciona bien cuando los clusters están bien separados.

Limitaciones

- El codo a veces es ambiguo y no hay un quiebre claro.
- La elección final tiene una cuota de subjetividad.
- Conviene complementarlo con el coeficiente de Silhouette para validar el k elegido.

SECCIÓN

Clustering jerárquico

04

Clustering jerárquico

- Método de aprendizaje no supervisado que agrupa los datos de forma jerárquica, creando una estructura tipo árbol llamada dendrograma.
- Dos enfoques principales:
 - **Aglomerativo (bottom-up):**
 - Cada punto inicia como un cluster individual.
 - Los clusters más cercanos se van fusionando hasta formar uno solo.
 - **Divisivo (top-down):**
 - Comienza con todos los puntos en un único cluster.
 - Se van dividiendo progresivamente en subgrupos más pequeños.

Metodología — Aglomerativo

- 1 Se parte de n clusters (cada observación es un cluster).
- 2 Se calcula la matriz de distancias (o similitudes) entre todos los clusters.
- 3 Se unen los dos clusters más cercanos.
- 4 Se repiten los pasos 2 y 3 hasta que todas las observaciones forman un único cluster.

Metodología — Aglomerativo

Vamos a clusterizar este dataset usando el método jerárquico aglomerativo.

$$d(A, B) = \sqrt{(0.0 - 0.2)^2 + (0.0 - 0.1)^2 + (0.0 - 0.0)^2} = \sqrt{0.04 + 0.01 + 0} = \sqrt{0.05} \approx 0.22$$

obs	x1	x2	x3
A	0.0	0.0	0.0
B	0.2	0.1	0.0
C	0.1	0.3	0.1
D	3.3	3.1	3.0
E	3.2	2.9	2.8
F	6.1	0.1	0.2
G	6.3	0.2	0.1

Metodología — Aglomerativo

Matriz de distancias inicial

$$d(A, B) = \sqrt{(0.0 - 0.2)^2 + (0.0 - 0.1)^2 + (0.0 - 0.0)^2} = \sqrt{0.04 + 0.01 + 0} = \sqrt{0.05} \approx 0.22$$



	A	B	C	D	E	F	G
A	0.00	0.22	0.33	5.25	5.15	6.00	6.30
B	0.22	0.00	0.24	5.08	4.97	5.80	6.10
C	0.33	0.24	0.00	4.97	4.86	5.90	6.20
D	5.25	5.08	4.97	0.00	0.35	5.08	5.26
E	5.15	4.97	4.86	0.35	0.00	4.74	4.92
F	6.00	5.80	5.90	5.08	4.74	0.00	0.33
G	6.30	6.10	6.20	5.26	4.92	0.33	0.00

Metodología — Aglomerativo

Uno los clusters A y B porque son los más cercanos y recalculo la matriz de distancias entre clusters.

	{C}	{D}	{E}	{F}	{G}	{A,B}
{C}	0.00	4.97	4.86	5.90	6.20	0.24
{D}	4.97	0.00	0.35	5.08	5.26	5.08
{E}	4.86	0.35	0.00	4.74	4.92	4.97
{F}	5.90	5.08	4.74	0.00	0.33	5.80
{G}	6.20	5.26	4.92	0.33	0.00	6.10
{A,B}	0.24	5.08	4.97	5.80	6.10	0.00

Cuando A y B se fusionaron en el cluster {A, B}, dejamos de tener distancias individuales de A y de B hacia el resto: ahora hay que definir una única distancia desde el cluster entero {A, B} hacia cada otro cluster.

Single linkage dice que esa distancia **es la mínima entre todos los pares posibles formados por un miembro de un cluster y un miembro del otro**. En otras palabras, dos clusters están "a la distancia de sus dos puntos más cercanos".

Clustering jerárquico

Metodología — Aglomerativo

Uno los clusters A-B y C, recalculo distancias entre clusters.

	{D}	{E}	{F}	{G}	{C,A,B}
{D}	0.00	0.35	5.08	5.26	4.97
{E}	0.35	0.00	4.74	4.92	4.86
{F}	5.08	4.74	0.00	0.33	5.80
{G}	5.26	4.92	0.33	0.00	6.10
{C,A,B}	4.97	4.86	5.80	6.10	0.00

Metodología — Aglomerativo

Uno los clusters F y G, recalculo distancias entre clusters.

	{D}	{E}	{C,A,B}	{F,G}
{D}	0.00	0.35	4.97	5.08
{E}	0.35	0.00	4.86	4.74
{C,A,B}	4.97	4.86	0.00	5.80
{F,G}	5.08	4.74	5.80	0.00

Metodología — Aglomerativo

Uno los clusters F y G, recalculo distancias entre clusters.

	{D}	{E}	{C,A,B}	{F,G}
{D}	0.00	0.35	4.97	5.08
{E}	0.35	0.00	4.86	4.74
{C,A,B}	4.97	4.86	0.00	5.80
{F,G}	5.08	4.74	5.80	0.00

Metodología — Aglomerativo

Uno los clusters D y E, recalculo distancias entre clusters.

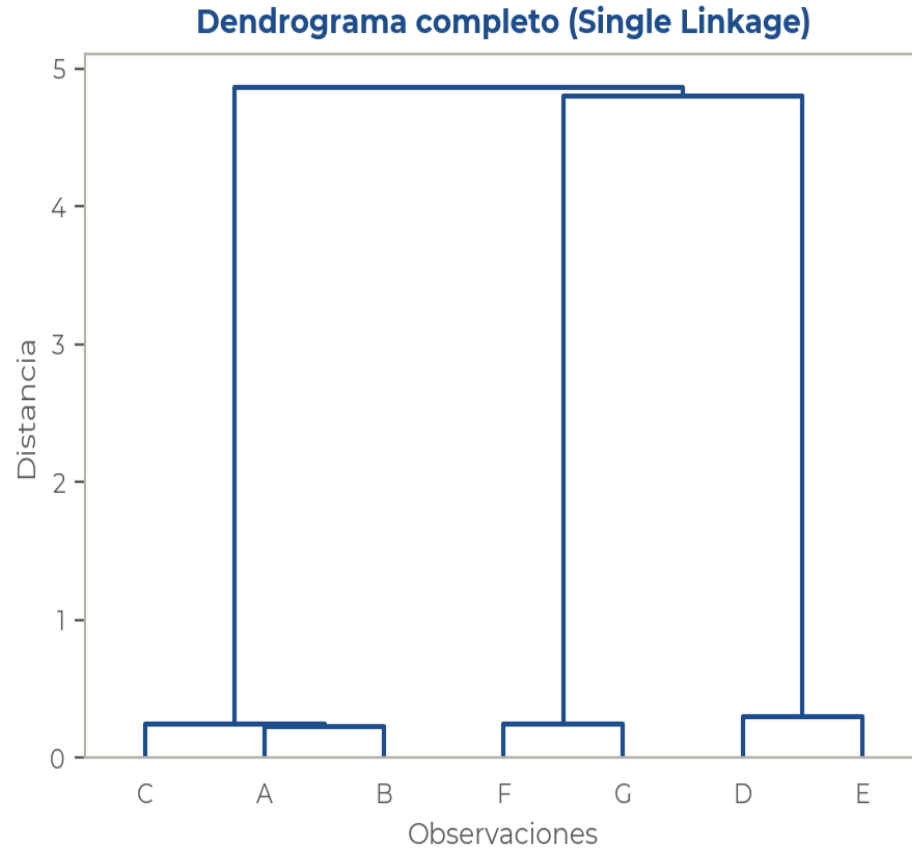
	{C,A,B}	{F,G}	{D,E}
{C,A,B}	0.00	5.80	4.86
{F,G}	5.80	0.00	4.74
{D,E}	4.86	4.74	0.00

Metodología — Aglomerativo

Uno los clusters F-G y D-E, recalculo distancias entre clusters.

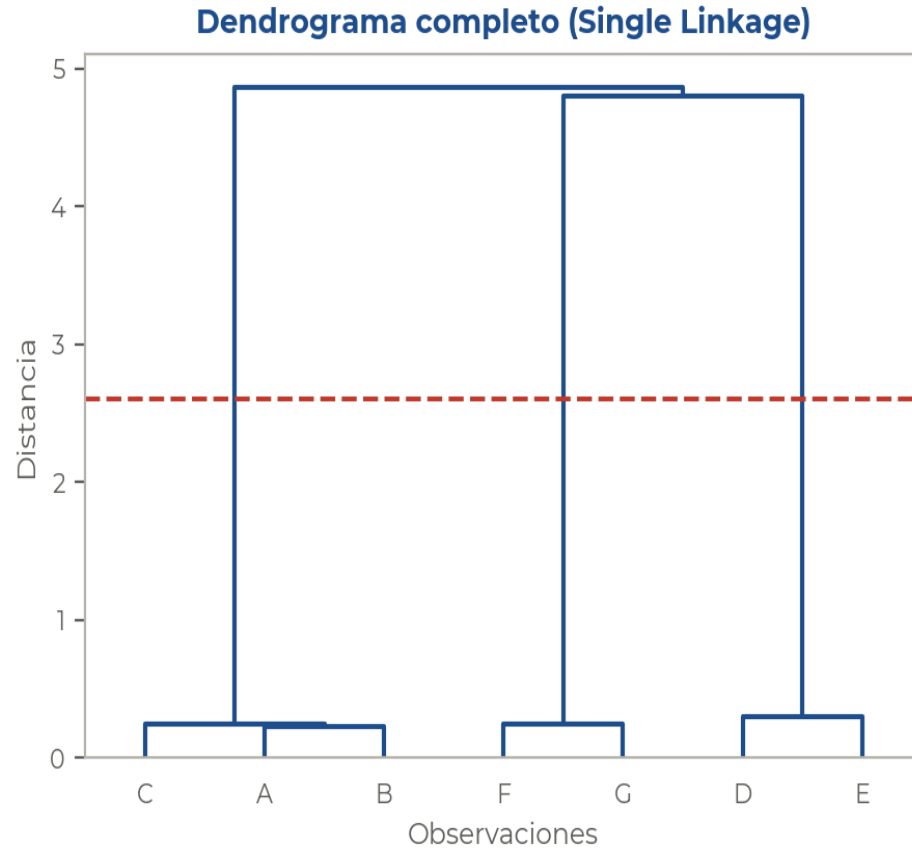
	{C,A,B}	{F,G,D,E}
{C,A,B}	0.00	4.86
{F,G,D,E}	4.86	0.00

Metodología — Aglomerativo



Uno los últimos dos clusters y grafico el dendrograma.

Metodología — Aglomerativo



La cantidad óptima de clusters se determina en la altura máxima entre dos iteraciones consecutivas.

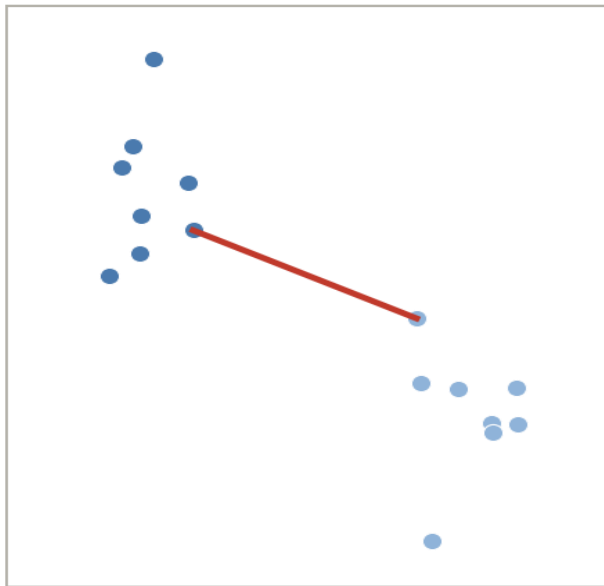
Distancias entre clusters

Alternativas para calcular la distancia:

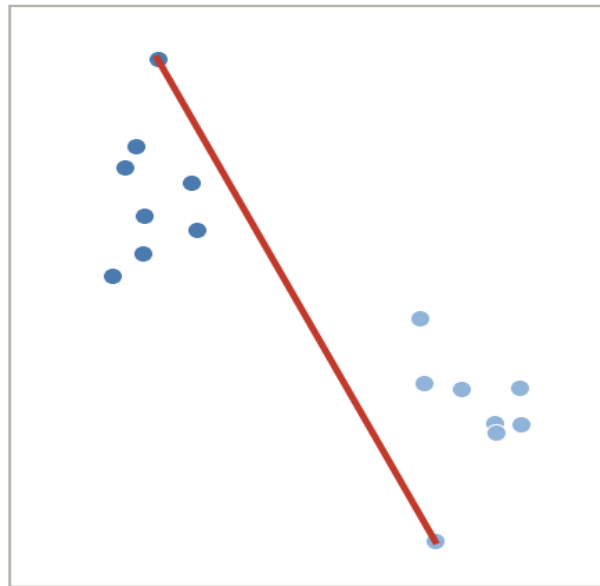
- Single linkage: se tiene en cuenta la mínima distancia entre clusters.
- Complete linkage: se tiene en cuenta la máxima distancia entre clusters.
- Mean linkage: se tiene en cuenta la distancia promedio entre clusters.

Distancias entre clusters

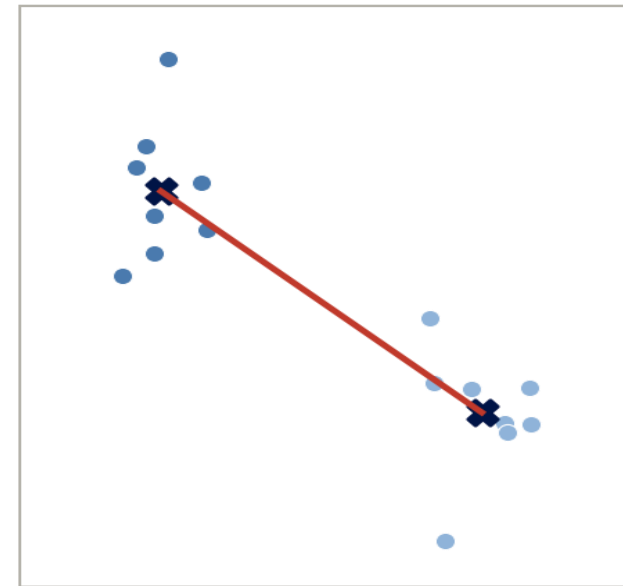
single-link



complete-link



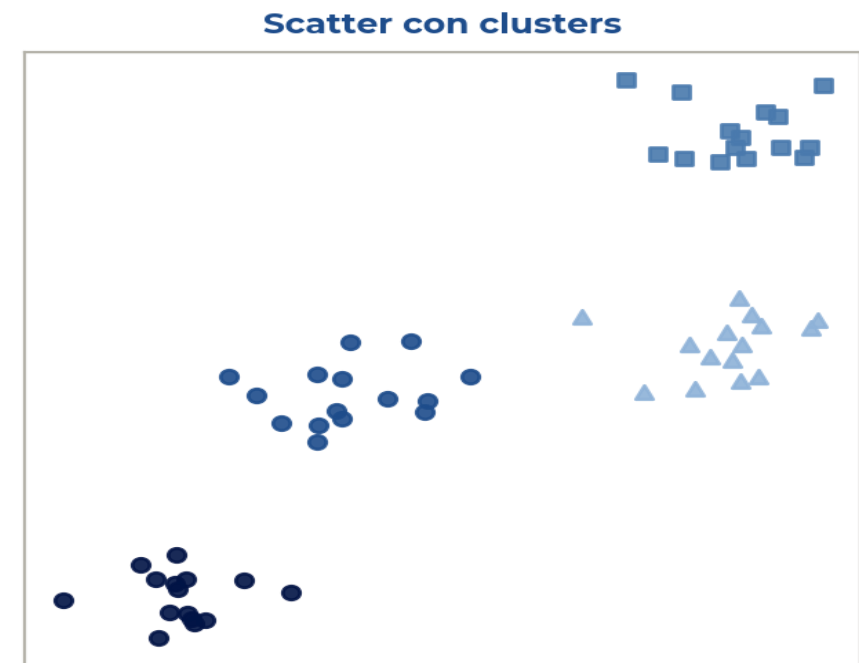
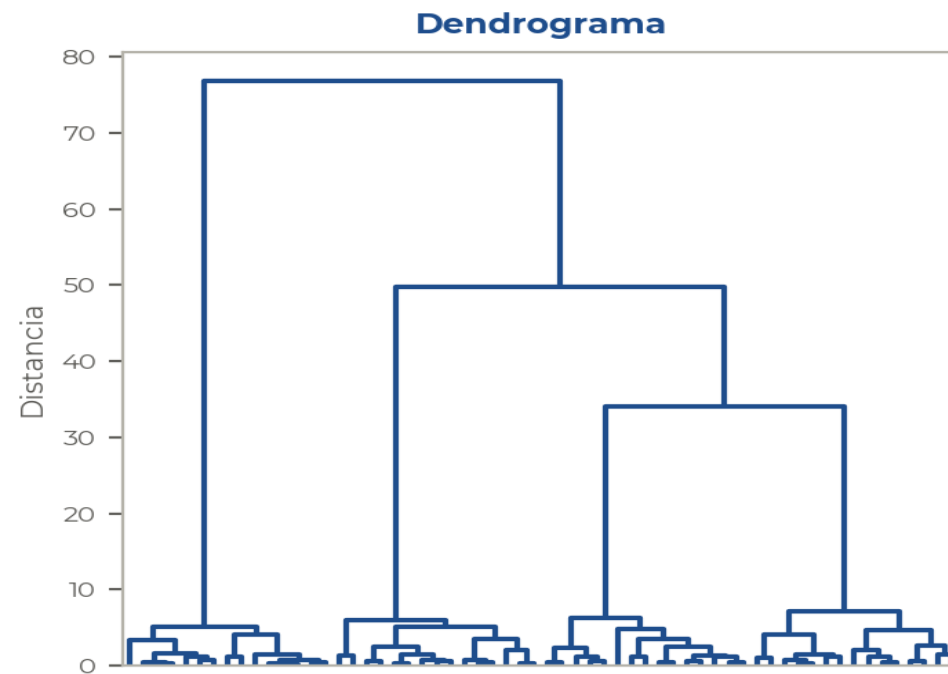
average-link



Clustering jerárquico

Clustering jerárquico

Dendrograma y scatter con los clusters formados.



Clustering jerárquico

Ventajas

- No requiere definir k al inicio.
- El dendrograma ofrece una visión jerárquica completa de las relaciones y del tamaño de los clusters.
- Permite elegir el número de clusters cortando el dendrograma.

Desventajas

- Alto costo computacional en grandes datasets.
- No permite “corregir” fusiones o divisiones previas: el dendrograma depende críticamente de las primeras fusiones.

SECCIÓN

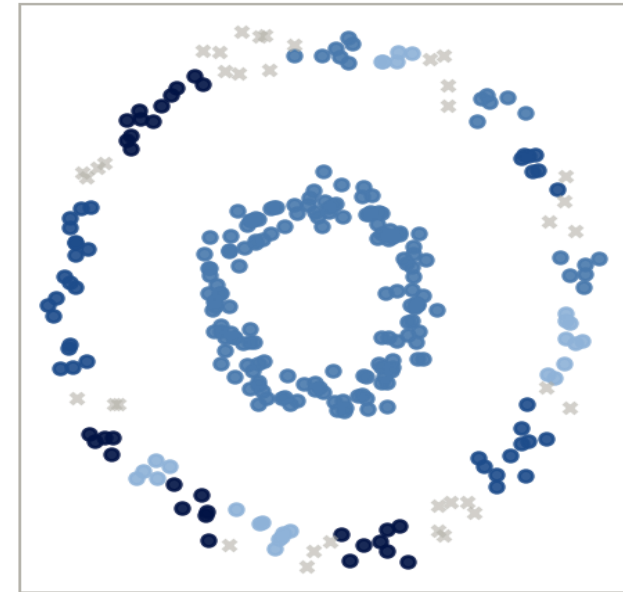
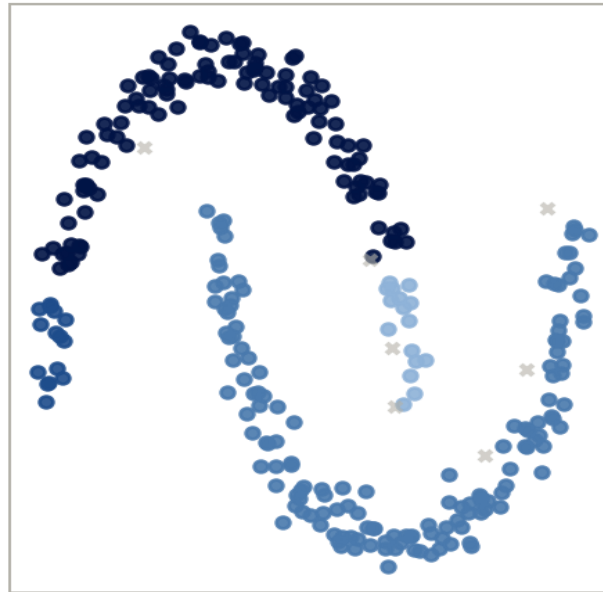
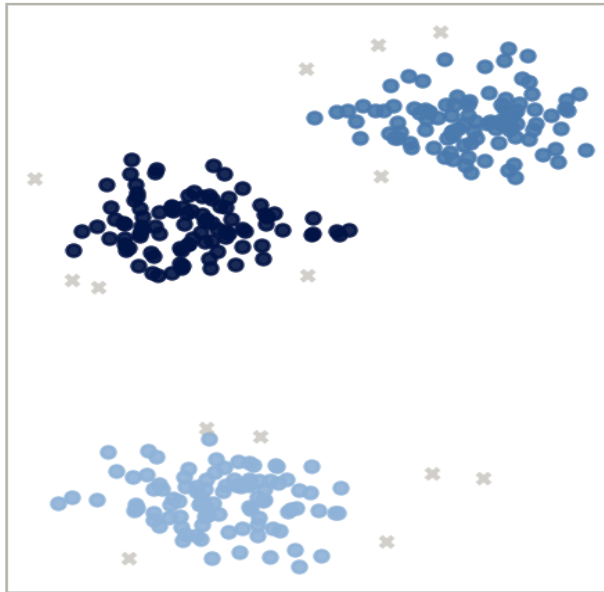
Otros métodos

05

DBScan

- **Density-Based Spatial Clustering of Applications with Noise.**
- Algoritmo de aprendizaje no supervisado que agrupa puntos cercanos entre sí según una medida de densidad.
- No requiere especificar el número de clusters previamente.
- Identifica ruido u outliers (puntos aislados).

DBScan



DBScan

ϵ (epsilon)

- Distancia máxima entre dos puntos para considerarse vecinos.

MinPts

- Número mínimo de puntos necesarios para formar una región densa.

DBScan — Tipos de puntos

Core points

- Tienen al menos MinPts vecinos dentro de ϵ .

Border points

- Están dentro de ϵ de un core point, pero no tienen suficiente densidad para ser core.

Noise

- Puntos que no pertenecen a ningún cluster.

DBScan

Ventajas

- Detecta clusters de forma arbitraria (no sólo esféricos).
- Resistente a ruido y valores atípicos.
- No depende de la inicialización aleatoria como k-means.

Desventajas

- Sensible a la elección de ϵ y MinPts.
- Dificultad para manejar datos con densidades muy diferentes.

C I E R R E

¡Gracias!

Analítica de Datos — Clustering

Pablo Sciolla

Universidad de San Andrés — Analítica de Datos